Lattice-Embedding + Tiling Discussion

Algorithms and Implementation

ABHIRUP MUKHERJEE

IISER KOLKATA

Emergent Phenomena in Quantum Matter group

IISER Kolkata

EPQM

Renormalisation Group Equation Simulation

Define Some Constants

```
const K_MIN = -pi
const K_MAX = pi
const TOLERANCE = 1e-8
const HOP_T = 1.0
const OMEGA_BY_t = -2 * HOP_T
const NODAL_POINTS = [(-π/2, -π/2), (-π/2, π/2), (π/2, π/2), (π/2, -π/2)]
const ANTINODAL_POINTS = [(-π, 0.), (0., π), (π, 0.), (0., -π)]
const RG_RELEVANCE_TOL = 1e-3
```

- Keep unchanged throughout all calculations
- Try to keep them dimensionless

Working In Momentum Space: Variables

- Each $(k_x, k_y) \to m$
- Store single momentum-dependent variable as vector. Dispersion, density of states:

$$E_{k_x, k_y} \longrightarrow E[m]$$

- Store two momentum-dependent variable as matrix. Kondo coupling:

$$J_{k_1, k_2} \longrightarrow J[m, n]$$

- Store four momentum-dependent variable as function. Call when needed. Bath interaction:

$$W_{k_1, k_2, k_3, k_4} \longrightarrow W(...k)$$

Working In Momentum Space: Helpers

- `map1DTo2D(m)` ⟶ `(kx, ky)`
- `map2DTo1D(kx, ky)` ⟶ `m`
- `getIsoEngCont(dispersion, E)` ⟶ `[m1, m2, ..., mN]`

(loop through points, check which energies are equal to probe energy upto some tolerance)

Density Of States

- Dispersion is easy

```
E[m] = Dispersion(kx, ky)
```

- DOS:

```
rho[m] ~ 1/Delta(E[m]) ~ 1/(E[m+1] - E[m])
```

- Take care of infinities (replace with largest finite value?!)
- Normalise per some convention to keep things bounded

$$\int dE \rho(E) \sim N$$

RG Equation

$$\Delta J[k_1, k_2] \sim \sum_q \left(J[k_1, q] J[q, k_2] - 4 J[q, \bar{q}] W[k_1, k_2]\right) G[q]$$

Write as matrix equation for vectorisation:

$$\Delta J \sim J G_{\mathrm{UV}} J^\dagger - 4 W \; \mathrm{Tr}\left(\bar{J} G_{\mathrm{UV}}\right)$$

where

$$J \longrightarrow J[k_1, k_2], \quad G_{\mathrm{UV}} \longrightarrow G[q], \quad W \longrightarrow W[k_1, k_2], \quad \bar{J} = J[q, \bar{q}]$$

RG Flow Simulation: Initialisation

$$\Delta J \sim J G_{\mathrm{UV}} J^\dagger - 4W\ \mathrm{Tr}\left(\overline{J} G_{\mathrm{UV}}\right)$$

- Define dispersion and density of states
- Define bath interaction matrix and initial Kondo matrix
- Extract cutoff energies from dispersion (by varying k_x from 0 $\longrightarrow \pi$ keeping $k_y = 0$)

RG Flow Simulation: Iteration

$$\Delta J \sim J G_{\mathrm{UV}} J^{\dagger} - 4W\ \mathrm{Tr}\left(\bar{J} G_{\mathrm{UV}}\right)$$

At each step,

- extract UV and IR momentum points
- define propagator diagonal matrix G by setting only UV points

```
GMatrix[q, q] = densityOfStates[q] ./ (OMEGA_BY_t * HOP_T - energyCutoff / 2
                                       + kondoJArray[q, q] / 4 + bathW / 2)
```

- use matrix multiplication to calculate ΔJ

```
delta = -abs(deltaEnergy) * (J[IR, UV] * G[UV, UV] * J[UV, IR]
                            .- 4 * bathW * traceGprime .* WMatrix[IR, IR])
```

(Use broadcasting to make the syntax readable)

Notes

- Parallelisation does not really help here
- Using reflection and particle-hole symmetries might help but makes code complicated (measure tradeoffs before finalising)
- Choosing system size as $N = 4m + 5$ helps (makes sure nodes, antinodes and midway point is available)

Thank You